TASK: 1

Implementation of Graph Search Algorithms

(Breadth First Search and Depth First Search)

Aim:

To implement graph search algorithms (Breadth First Search and Depth First Search) using Python.

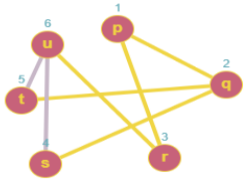
Task 1A: Breadth First Search (BFS)

Algorithm:

1. Start by putting any one of the graph's vertices at the back of the queue.
2. Now take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add those which are not within the visited list to the rear of the queue.
4. Keep continuing steps two and three till the queue is empty.

Program:

```
from collections import deque

def bfs(graph, start):
queue, visited = deque([start]), set()
print("BFS:", end=" ")
while queue:
node = queue.popleft()
if node not in visited:
print(node, end=" ")
visited.add(node)
queue.extend(neighbor for neighbor in graph[node] if neighbor not in visited)
print()

# Example graph
graph = {
'P': ['Q', 'R'],
'Q': ['P', 'S', 'T'],
'R': ['P', 'U'],
'S': ['Q'],
'T': ['Q', 'U'],
'U': ['R', 'T']
bfs(graph, 'P')
```



Output:

```
BFS: P Q R S T U
```

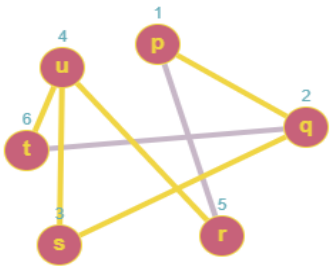
Task 1B: Depth First Search (DFS)

Algorithm:

1. Declare a stack and insert the starting Vertex.
2. Initialize a visited set and mark the starting Vertex as visited.
3. Remove the top vertex from the stack.
4. Mark that vertex as visited.
5. Insert all the unvisited neighbors of the vertex into the stack.
6. Repeat until the stack is empty.

Program:

```
def dfs(graph, start):
stack, visited = [start], set()
print("DFS:", end=" ")
while stack:
node = stack.pop()
if node not in visited:
print(node, end=" ")
visited.add(node)
stack.extend(reversed([neighbor for neighbor in graph[node] if neighbor not
in visited]))
print()

# Example graph
graph = {
'P': ['Q', 'R'],
'Q': ['P', 'S', 'T'],
'R': ['P', 'U'],
'S': ['Q'],
'T': ['Q', 'U'],
'U': ['R', 'T']
}

dfs(graph, 'P')
```



Output:

```
DFS: P R U T Q S
```

Result:

Thus, the implementation of Graph Search Algorithms (Breadth First Search and Depth First Search) using Python was successfully executed, and the output was verified.

TASK: 2

Implementation of Hill Climbing Algorithm for Heuristic Search Approach

---

Aim:

To implement the Hill Climbing algorithm for the Heuristic search approach to solve the Travelling Salesman Problem (TSP) using Python.

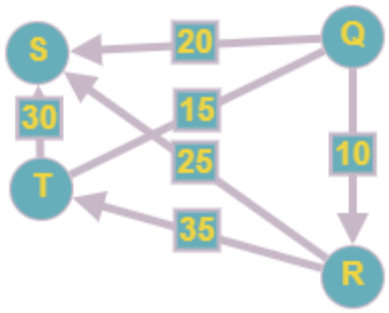
Algorithm:

1. Start.
2. Define TSP with `(graph, s)` and assign values for vertices.
3. Store all vertices apart from the source vertex.
4. Store the minimum weight Hamiltonian cycle and assign permutations (vertex).
5. Store current path weight (cost) and compute the current path weight.
6. Update minimum and matrix representation of the graph values and print it.
7. Stop.

Program:

```
from sys import maxsize
from itertools import permutations

V = 4

def travellingSalesmanProblem(graph, s):
vertex = []  # Store all vertices except source
for i in range(V):
if i != s:
vertex.append(i)

min_path = maxsize  # Initialize minimum path weight to maximum possible

# Generate all permutations of vertices to calculate all possible paths
next_permutation = permutations(vertex)

for i in next_permutation:
current_pathweight = 0
k = s
```

```
for j in i:
current_pathweight += graph[k][j]
k = j
current_pathweight += graph[k][s]

min_path = min(min_path, current_pathweight)

return min_path

if __name__ == "__main__":
graph = [
[0, 10, 15, 20],
[10, 0, 35, 25],
[15, 35, 0, 30],
[20, 25, 30, 0]
]
s = 0
print(travellingSalesmanProblem(graph, s))
```



Output:

```
80
```

Result:

Thus, the implementation of the Hill Climbing algorithm for the heuristic search approach to the Travelling Salesman Problem using Python was successfully executed and the output was verified.

TASK: 3

Implementation of A* Algorithm to Find the Optimal Path Using Python

Aim:

To implement the A* algorithm to find the optimal path using Python (suitable for Jupyter Notebook).

3(A) A* Algorithm

Algorithm:

1. Start.
2. Place the starting node into the open set and compute its f(n).
3. Remove the node from the open set with the lowest f(n). If it is the goal node, stop and return the path.
4. Otherwise, expand its neighbors, calculate their f(n), add them to open set, and move the current node to the closed set.
5. Repeat until the goal is reached or no nodes remain.
6. Exit.

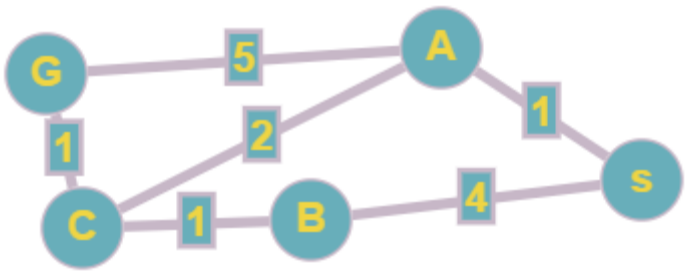
Simplified Program:

```
def a_star(start, goal):
    open_set = {start}
    closed_set = set()
    g = {start: 0}
    parents = {start: None}

    def heuristic(n):
        h = {'S': 7, 'A': 6, 'B': 2, 'C': 1, 'G': 0}
        return h.get(n, 0)

    graph = {
        'S': [('A', 1), ('B', 4)],
        'A': [('S', 1), ('C', 2), ('G', 5)],
        'B': [('S', 4), ('C', 1)],
        'C': [('A', 2), ('B', 1), ('G', 1)],
        'G': []
    }

    while open_set:
        n = min(open_set, key=lambda v: g[v] + heuristic(v))

        if n == goal:
            path = []
```

```
            while n:
                path.append(n)
                n = parents[n]
            print("Path found:", path[::-1])
            return path[::-1]

        open_set.remove(n)
        closed_set.add(n)

        for (neighbor, cost) in graph.get(n, []):
            if neighbor in closed_set:
                continue
            tentative_g = g[n] + cost
            if neighbor not in open_set or tentative_g < g.get(neighbor,
float('inf')):
                parents[neighbor] = n
                g[neighbor] = tentative_g
                open_set.add(neighbor)

    print("Path does not exist!")
    return None

print("Running A* Algorithm:")
a_star('S', 'G')
```

Output:

```
Path found: ['S', 'A', 'C', 'G']
```



3(B) Simplified A* Algorithm

Aim:

To implement a simplified A* algorithm for pathfinding.

Algorithm:

- Same as above with minimal complexity.

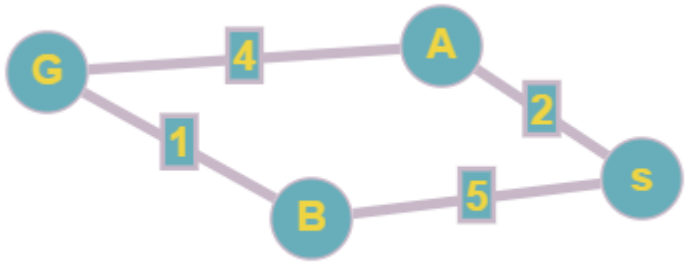
Simplified Program:

```
def simplified_a_star(start, goal):
    open_set = {start}
    closed_set = set()
    g = {start: 0}
    parents = {start: None}

    def heuristic(n):
        h = {'S': 5, 'A': 4, 'B': 2, 'G': 0}
        return h.get(n, 0)

    graph = {
        'S': [('A', 2), ('B', 5)],
        'A': [('S', 2), ('G', 4)],
        'B': [('S', 5), ('G', 1)],
        'G': []
    }

    while open_set:
        n = min(open_set, key=lambda v: g[v] + heuristic(v))

        if n == goal:
            path = []
            while n:
                path.append(n)
                n = parents[n]
            print("Path found:", path[::-1])
            return path[::-1]

        open_set.remove(n)
        closed_set.add(n)

        for (neighbor, cost) in graph.get(n, []):
            if neighbor in closed_set:
                continue
            tentative_g = g[n] + cost
            if neighbor not in open_set or tentative_g < g.get(neighbor,
float('inf')):
                parents[neighbor] = n
                g[neighbor] = tentative_g
                open_set.add(neighbor)
```

```
    print("Path does not exist!")
    return None

print("Running Simplified A* Algorithm:")
simplified_a_star('S', 'G')
```

Output:

```
Path found: ['S', 'B', 'G']
```



Result:

Thus, the implementation of both A* algorithms to find the optimal path using Python was successfully executed and the output was verified.

TASK: 6

Solve a Map Coloring Problem Using Constraint Satisfaction Approach

---

Aim:

To solve a Map Coloring problem using the constraint satisfaction approach by applying the following constraints.

Algorithm:

1. Confirm whether it is valid to color the current vertex with the current color by checking whether any of its adjacent vertices are colored with the same color.
2. If yes, then color it; otherwise, try a different color.
3. Check if all vertices are colored or not.
4. If not, then move to the next adjacent uncolored vertex.
5. Backtracking means stopping further recursive calls on adjacent vertices.

Program:

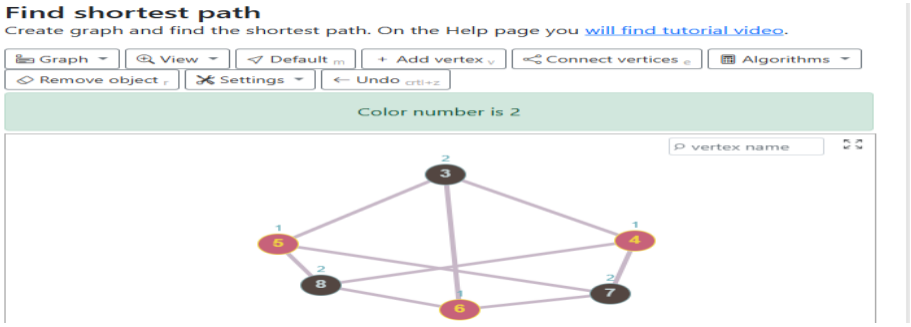
```
class Graph:
def __init__(self, vertices):
self.v = vertices
self.graph = [[0 for column in range(vertices)] for row in range(vertices)]

# A utility function to check if the current color assignment is safe for
vertex v
def is_safe(self, v, color, c):
for i in range(self.v):
if self.graph[v][i] == 1 and color[i] == c:
return False
return True

# A recursive utility function to solve m-coloring problem
def graph_color_util(self, m, color, v):
if v == self.v:
return True
for c in range(1, m + 1):
if self.is_safe(v, color, c):
color[v] = c
if self.graph_color_util(m, color, v + 1):
```

```
return True
color[v] = 0
return False

def graph_coloring(self, m):
color = [0] * self.v
if not self.graph_color_util(m, color, 0):
return False
# Print the solution
print("Solution exists and following are the assigned colors:")
for c in color:
print(c, end=" ")
```

Driver Code:

```
if __name__ == '__main__':
g = Graph(4)
g.graph = [
[0, 1, 1, 1],
[1, 0, 1, 0],
[1, 1, 0, 1],
[1, 0, 1, 0]
]
m = 3
# Function call
g.graph_coloring(m)
```

Output:




```
Solution exists and following are the assigned colors:
1 2 3 2
```

Result:

Thus, solving a Map Coloring problem using the constraint satisfaction approach with Graphonline and Visualago online simulators was successfully executed, and the output was verified.